Nika ERP Backend - Code Explanation and Audit
Generated: 2026-10-13

1) Repository Summary
This repository contains an AI-powered ERP invoice processing system with:
- A FastAPI backend for OCR/AI extraction, workspace/account controls, invoice storage, and auditing.
- A React + TypeScript frontend for upload, dashboards, reports, and ledger editing.
- Utility scripts to generate synthetic invoice test data.

Top-level important files/folders:
- /home/runner/work/nika-erp-backend/nika-erp-backend/generated_server.py (main production-style back
- /home/runner/work/nika-erp-backend/nika-erp-backend/server.py (simpler prototype backend)
- /home/runner/work/nika-erp-backend/nika-erp-backend/ai-accounting-ui (frontend app)
- /home/runner/work/nika-erp-backend/nika-erp-backend/generate_tests.py (synthetic invoice dataset gen

2) Backend Code Explanation (generated_server.py)
2.1 Tech stack and bootstrapping
- FastAPI app with CORS enabled.
- SQLAlchemy ORM models and session management.
- Clerk token verification using JWKS for user auth.
- OpenRouter API integration for vision LLM invoice extraction.

2.2 Data model
- Organization: tenant/workspace with API credit balance.
- User: identity record.
- OrganizationMember: maps user to organization role.
- Invoice: main invoice record (org-scoped).
- InvoiceItem: line items linked to invoice.
- ApiKey: machine-to-machine key metadata (hashed key only in DB).
- AuditLog: immutable edit trail for changed invoice fields.

2.3 Authentication and authorization
- A single auth dependency allows either:
  A) Machine key path: token prefix nk_live_, validated by sha256 hash lookup.
  B) Human path: Clerk JWT verified via RS256 signing key from JWKS endpoint.
- After auth, request context carries org_id and caller identity.
- Every invoice query/update is filtered by org_id to enforce tenant isolation.

2.4 Auto-provisioning flow
- If a new Clerk user is seen, code auto-creates user + organization + owner mapping.
- This makes first-login onboarding zero-touch.

2.5 Image optimization and AI extraction
- Uploaded image is resized/compressed via Pillow (max 1024x1024, JPEG quality 85).
- Image is base64-embedded into OpenRouter request.
- LLM is prompted to return strict JSON with vendor, invoice details, line items, totals, and confidence metadata.
- API usage tokens are converted to internal credit deduction.

2.6 Business rules and invoice write path
- Rejects if organization has no credits.
- Rejects files larger than 10MB.
- Runs extraction, computes required credits, blocks if insufficient.
- Validates math by comparing grand total vs line-item total.
- Uses confidence threshold (>=85) and math validity to assign status:
  Pending or Needs Review.
- Persists invoice and line items transactionally.

2.7 API endpoints

- POST /upload-invoice/: upload + AI extraction + persistence + credit deduction.
- GET /invoices/: paginated org-scoped invoice history with items and audit logs.
- PUT /invoices/{invoice_id}: editable invoice updates + audit log entries.
- GET /workspace/: returns workspace/credits summary.
- POST /api-keys/: generates machine API keys for authenticated human users.

3) Prototype Backend (server.py)
- Minimal upload endpoint calling OpenRouter for basic invoice fields.
- Looks like an earlier prototype beside generated_server.py.

4) Frontend Code Explanation (ai-accounting-ui)
4.1 App shell and auth
- ClerkProvider wraps app in main.tsx.
- App.tsx uses SignedIn/SignedOut guards for secure workspace access.

4.2 Main modules
- HomeTab: module launcher cards.
- UploadTab: multi-file queue processing with progress and per-file result cards.
- DashboardsTab: dashboard creation, report pinning, chart previews.
- ReportsTab: custom report builder with filters, chart type switching, and column toggles.
- InvoicesTab: ledger table with sorting/filtering, deep edit mode, pagination, and CSV export.
- SettingsTab: credit overview + machine API key generation UX.

4.3 Data flow
- Frontend calls hosted API with bearer tokens from Clerk.
- Reports and dashboards are persisted in browser localStorage.
- CSV exports are generated client-side.

5) Test/Data Utility
- generate_tests.py synthesizes invoice images and matching JSON answer keys.
- test_data/images and test_data/answers hold generated datasets.

6) Audit Findings
Severity: High / Medium / Low

High
1. SQL inspector concept is high-risk.
   - InspectorTab sends raw SQL query text to backend endpoint /api/inspector/query.
   - If implemented without strict controls, this can become a critical data exposure/RCE vector.

2. CORS policy is fully open (*).
   - Should be restricted to trusted frontend domains in production.

Medium
3. Duplicate InvoiceItemEdit class declaration in generated_server.py.
   - Indicates maintainability issue and likely merge artifact.

4. UI routing inconsistency for inspector.
   - Sidebar includes "SQL Inspector" but App.tsx does not render InspectorTab.

5. Lack of visible automated API/security test suite.
   - Synthetic data generation exists, but endpoint-level testing is not evident in reviewed files.

Low
6. Multiple backend entry files may cause deployment confusion.
   - server.py and generated_server.py overlap in responsibility.

7) Recommended Improvements
1. Remove or harden SQL inspector (role-based, read-only, allowlisted queries only).
2. Restrict CORS per environment.

3. Remove duplicate model declarations.
4. Align sidebar navigation with mounted tabs.
5. Add automated tests for auth, tenant isolation, upload path, and audit logging.
6. Document the single authoritative backend entrypoint.

8) Overall Assessment
- Strong product foundation: AI ingestion + editable ledger + analytics/reporting.
- Main production readiness gaps are security hardening, consistency cleanup, and test coverage.